



CP Bootcamp 2026 — Day 9

Theme: Two Pointer + Sliding Window Foundation

Main Goal:

আজ থেকে তুমি একটা বড় optimization idea শিখবে।

এতদিন:

Nested Loop

+

Check All Possibilities

করেছো।

এখন:

Observe Pattern

↓

Avoid Repeated Work

↓

Move Pointers / Window

↓

$O(n)$ Solution



Day 9 Chapter Map

Day 9

- Chapter 1 – Why Optimization Patterns Matter
- Chapter 2 – Two Pointer Mental Model
- Chapter 3 – Two Pointer on Sorted Arrays
- Chapter 4 – Opposite Direction Two Pointer
- Chapter 5 – Same Direction Two Pointer
- Chapter 6 – Sliding Window Mental Model
- Chapter 7 – Fixed Size Sliding Window
- Chapter 8 – Variable Size Sliding Window
- Chapter 9 – Common Bugs & Edge Cases
- Chapter 10 – Day 9 Assignment & Practice

Chapter 1 — Why Optimization Patterns Matter

ধরো problem:

Array-তে দুইটি number-এর sum target কিনা বের করো।

Array:

2 4 6 8 10

Target:

12

Beginner Approach

সব pair check:

```
for(i=0;i<n;i++)
{
    for(j=i+1;j<n;j++)
    {
        if(arr[i]+arr[j]==target)
        {
            found=1;
        }
    }
}
```

Complexity:

$O(n^2)$

যদি:

$n = 100000$

তাহলে:

10,000,000,000

বার check!

Impossible.

Optimization idea:

Can we use order?

যদি array sorted হয়:

2 4 6 8 10

তাহলে:

left right

2 4 6 8 10

↑ ↑

এখান থেকেই:

Two Pointer

Chapter 2 — Two Pointer Mental Model

Two Pointer মানে:

একই data structure-এর উপর দুইটি pointer দিয়ে কাজ করা।

Example:

Array:

1 3 5 7 9

Pointers:

left → শুরু

right → শেষ

Visualization:

```
left
↓
1 3 5 7 9
      ↑
      right
```

Pointers move করে:

left++

right--

কেন Two Pointer?

কারণ:

একই element বারবার check না করে:

Information ব্যবহার করে
Pointer move করা

Chapter 3 — Two Pointer on Sorted Arrays

Problem

Sorted array:

1 2 4 7 9

Target:

10

Find pair।

Start:

left = 1

right = 9

Sum:

1+9 = 10

Found!

আরেক example:

Array:

1 3 5 8 12

Target:

13

Step 1:

$1 + 12 = 13$

Found!

যখন Sum ছোট

Example:

$1 + 8 = 9$

target = 13

Need bigger number!

তাই:

left++

যখন Sum বড়

Example:

$$5 + 12 = 17$$

$$\text{target} = 13$$

Need smaller number।

তাই:

right--

Decision Rule

`sum < target`

`→ left++`

`sum > target`

`→ right--`

`sum == target`

`→ answer`

Implementation

```
int twoSum(int arr[], int n, int target)
{
    int left = 0;
    int right = n-1;

    while(left < right)
    {
        int sum = arr[left] + arr[right];

        if(sum == target)
        {
            return 1;
        }

        else if(sum < target)
        {
            left++;
        }

        else
        {
            right--;
        }
    }

    return 0;
}
```

Complexity:

$O(n)$

Chapter 4 — Opposite Direction Two Pointer

এই pattern:

Start

↓

←

→

End

Common uses:

1. Pair Sum

left + right

2. Reverse Array

Example:

Before:

1 2 3 4 5

Swap:

1 ↔ 5

2 ↔ 4

After:

5 4 3 2 1

Code:

```
void reverse(int arr[], int n)
{
    int left=0;
    int right=n-1;

    while(left<right)
    {
        int temp=arr[left];

        arr[left]=arr[right];

        arr[right]=temp;

        left++;

        right--;
    }
}
```

3. Palindrome Check

String:

madam

Compare:

m = m

a = a

d = d

Pattern:

Two Pointer

+

Comparison

Chapter 5 — Same Direction Two Pointer

এখানে দুই pointer একই দিকে যায়।

Example:

slow →

fast →

Common use:

- Remove duplicates
- Move zeros
- Filtering

Example — Remove Duplicates

Sorted array:

1 1 2 2 3

Need:

1 2 3

Slow:

Unique position

Fast:

Explore

Visualization:

```
slow
↓
1 1 2 2 3
↑
fast
```

Mental Model:

fast finds useful data

slow stores useful data

Chapter 6 — Sliding Window Mental Model

Sliding Window হলো Two Pointer-এর special case।

Problem:

Continuous part (subarray/substring) নিয়ে কাজ করা।

Example:

Array:

1 2 3 4 5

Window size:

3

First window:

1 2 3

Next:

2 3 4

Next:

3 4 5

Window slide করে।

Mental Model:

[window]

Move →

Move →

Move →

Why Sliding Window?

Without:

প্রতিটি subarray calculate:

$O(n^2)$

With window:

$O(n)$

Chapter 7 — Fixed Size Sliding Window

Problem:

Size k subarray-এর maximum sum বের করো।

Example:

Array:

2 3 1 5 4

k :

3

Windows:

2 3 1 = 6

3 1 5 = 9

1 5 4 = 10

Answer:

10

Naive

প্রতিবার sum:

$O(n*k)$

Sliding Window

First sum:

$2+3+1$

Next:

Remove:

2

Add:

5

New:

$3+1+5$

Pattern:

Old Window

-remove outgoing

+add incoming

New Window

Code:

```
int maxSum(int arr[], int n, int k)
{
    int sum=0;

    for(int i=0;i<k;i++)
    {
        sum+=arr[i];
    }

    int ans=sum;

    for(int i=k;i<n;i++)
    {
        sum += arr[i];

        sum -= arr[i-k];

        if(sum>ans)
            ans=sum;
    }

    return ans;
}
```

Chapter 8 — Variable Size Sliding Window

এখানে window size fixed না।

Condition অনুযায়ী:

Expand

Shrink

Example:

Smallest subarray whose sum $\geq$ target

Array:

2 3 1 2 4 3

Target:

7

Window:

2 3 1 2

sum:

8

Enough!

Shrink:

Remove left.

3 1 2

sum:

6

Not enough!

Expand again!

Mental Model:

Right pointer



Expand window

Condition true?



Left pointer



Shrink window

Common Problems

Variable Sliding Window:

Longest substring

Smallest subarray

Maximum length segment

Chapter 9 — Common Bugs & Edge Cases

Bug 1 — Two Pointer without Sorting

Wrong:

8 2 5 1 9

Two sum I

কারণ movement rule কাজ করবে না।

Need:

Sorted Data

Bug 2 — Wrong Condition

Pair sum:

Wrong:

```
while(left ≤ right)
```

Correct:

```
while(left < right)
```

কারণ:

এক element নিজে pair না।

Bug 3 — Sliding Window Remove Wrong Element

Window:

$i-k$

not:

$i-k+1$

Bug 4 — Forget Initial Window

Fixed window:

প্রথম k element আগে calculate করতে হবে।

Bug 5 — Empty Input

Handle:

$n=0$

Chapter 10 — Day 9 Assignment

Part A — Two Pointer Practice

Implement:

1. Reverse Array
2. Palindrome Check
3. Sorted Pair Sum
4. Count Pair Sum
5. Merge Two Sorted Arrays

Part B — Sliding Window

Implement:

1. Maximum Sum Subarray of Size K
2. Minimum Sum Subarray of Size K
3. Average of Every Window
4. Maximum Element in Every Window

Part C — Dry Run

Array:

2 3 1 5 4

k:

3

Complete:

Initial Window:

Sum:

Slide 1:

Remove:

Add:

New Sum:

Slide 2:

Remove:

Add:

New Sum:

Part D — Pattern Recognition

Problem 1

Reverse a string

Pattern:

?

Problem 2

Find pair with target sum in sorted array

Pattern:

?

Problem 3

Maximum sum of k consecutive elements

Pattern:

?

Problem 4

Longest substring without repeating character

Pattern:

?



Glossary Update

Add:

Two Pointer

Left Pointer

Right Pointer

Slow Pointer

Fast Pointer

Opposite Direction

Same Direction

Sliding Window

Fixed Window

Variable Window

Expand Window

Shrink Window

Continuous Segment

Subarray

Substring



Pattern Library Update

Add:

Two Pointer Pattern

Recognition:

- Sorted array
- Pair problems
- Reverse
- Palindrome

Mental Model:

left



data



right

Move pointers based on condition.

Sliding Window Pattern

Recognition:

- Continuous subarray
- Continuous substring
- Range problems

Mental Model:

Create Window

↓

Move Window

↓

Maintain Information

Types:

Fixed Size

Variable Size



Day 9 Completion Checklist

- [] Two Pointer idea বুঝি
- [] Sorted array-তে pair sum solve করতে পারি
- [] Reverse array two pointer দিয়ে করতে পারি
- [] Palindrome check করতে পারি
- [] Same direction pointer বুঝি
- [] Sliding Window বুঝি
- [] Fixed window solve করতে পারি
- [] Variable window idea বুঝি
- [] $O(n^2)$ থেকে $O(n)$ optimization বুঝি
- [] কখন Two Pointer use করবো জানি
- [] কখন Sliding Window use করবো জানি



Day 9 Final Mental Model

Repeated Work



Can I remember previous information?



Move Pointer



Maintain Window



$O(n)$ Solution

Day 9 শেষে তোমার CP foundation:

Day 1-5

Pattern Recognition

+

Day 6

Functions

+

Day 7

Searching

+

Day 8

Sorting + Complexity

+

Day 9

Optimization Patterns

পরের **Day 10 — Recursion + Backtracking Foundation** হবে।